LOW LEVEL DESIGN (LLD)

1. Module-Level Design

1.1 Document Processing Module

The Document Processing Module is responsible for ingesting the input PDF and extracting textual content. It utilizes a document loader to convert unstructured PDF data into structured text, which serves as the foundation for downstream processing.

Responsibilities:

- Load PDF documents
- Extract text content
- Clean and preprocess data

Input:

- PDF file

Output:

- Raw extracted text

1.2 Chunking Module

This module divides the extracted text into smaller, semantically meaningful chunks (typically 300–500 tokens). Chunking improves retrieval efficiency and ensures that the context passed to the language model remains relevant and precise.

Responsibilities:

- Split text into smaller chunks
- Maintain semantic continuity

Parameters:

- Chunk Size: 500 tokens
- Overlap: 50 tokens

Output:

- List of text chunks

1.3 Embedding Module

The Embedding Module transforms each text chunk into a high-dimensional vector representation using a pre-trained embedding model. These embeddings capture semantic relationships and enable similarity-based retrieval.

Responsibilities:

- Convert text chunks into vector embeddings

Input:

- Text chunks

Output:

- Vector embeddings

1.4 Vector Storage Module

This module stores the generated embeddings in the ChromaDB vector database along with associated metadata such as chunk identifiers and source references. It enables efficient indexing and retrieval operations.

Responsibilities:

- Store embeddings efficiently
- Perform similarity search

Data Stored:

- Chunk text
- Embedding vectors
- Metadata (chunk ID, source)

1.5 Retrieval Module

The Retrieval Module processes the user query by converting it into an embedding and performing a similarity search in the vector database. It retrieves the top-k most relevant chunks based on semantic similarity.

Responsibilities:

- Convert user query into embedding
- Retrieve top-k relevant chunks

Parameters:

- Top-k: 3–5

Output:

- Relevant document chunks

1.6 Query Processing Module

This module handles incoming user queries, performs validation and preprocessing, and forwards them to the graph execution module for further processing within the system workflow.

Responsibilities:

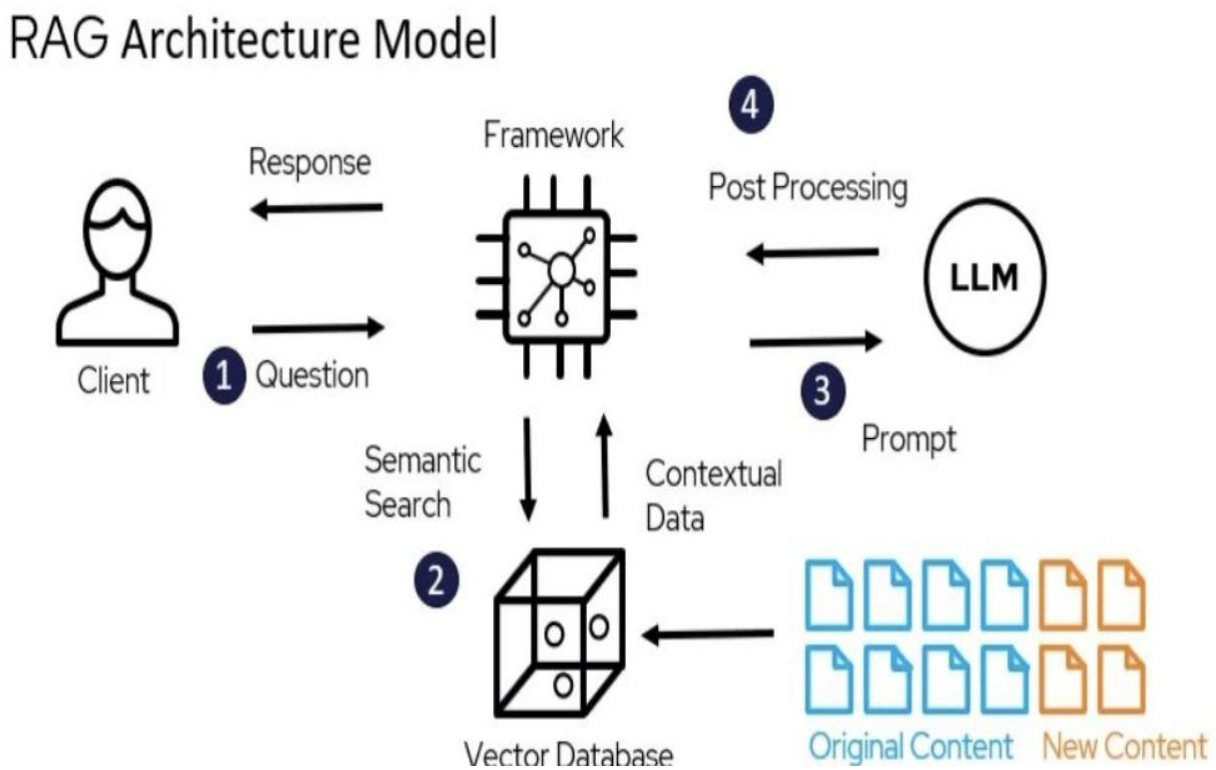
- Accept user input
- Normalize and preprocess query
- Pass query to retrieval system

1.7 Graph Execution Module (LangGraph)

The Graph Execution Module orchestrates the overall workflow using a graph-based structure. It manages state transitions, controls execution flow between components, and applies conditional routing logic to determine the appropriate response path.

Responsibilities:

- Generate contextual answers
- Use retrieved chunks as context



2. Data Structures

2.1 Document Representation

```
{
  "document_id": "doc_001",
  "content": "Extracted text from PDF document"
}
```

2.2 Chunk Representation

```
{
  "chunk_id": "chunk_001",
  "text": "Segment of document text",
  "document_id": "doc_001"
}
```

2.3 Embedding Structure

```
{
  "chunk_id": "chunk_001",
  "embedding": [0.123, 0.456, 0.789, ...]
}
```

2.4 Query-Response Schema

```
{
  "query": "What is the refund policy?",
  "retrieved_chunks": ["chunk_001", "chunk_010"],
  "response": "The refund policy allows returns within 30 days...",
  "confidence_score": 0.85,
  "escalation_flag": false
}
```

2.5 Graph State Object

```
{
  "query": "",
  "context": "",
  "response": "",
  "confidence": 0.0,
  "escalate": false
}
```

3. Workflow Design (LangGraph)

3.1 Nodes

- Processing Node: Executes query processing, retrieval, and response generation.
- Output Node: Produces the final response or triggers escalation.

3.2 Edges

- Input → Processing Node
- Processing Node → Output Node
- Processing Node → HITL Node (conditional path)

3.3 State Management

The system maintains a state object that flows across nodes and contains:

- User query
- Retrieved contextual data
- Generated response
- Confidence score
- Escalation indicator

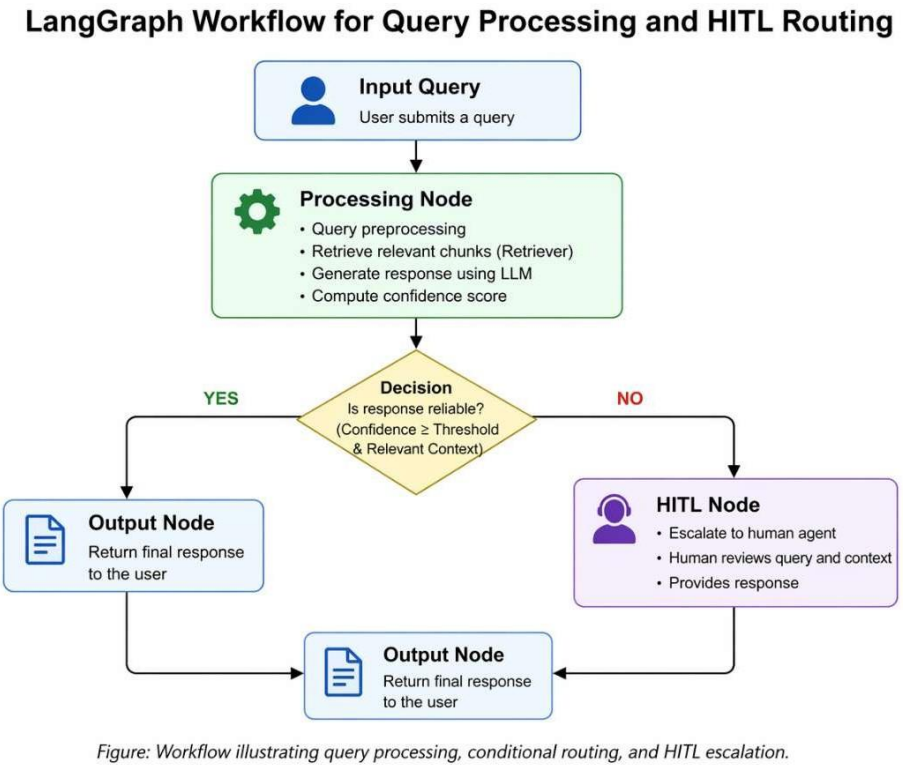


Figure: Workflow illustrating query processing, conditional routing, and HITL escalation.

4. Conditional Routing Logic

4.1 Answer Generation Criteria

- Relevant document chunks are successfully retrieved.
- The generated response is coherent and contextually accurate.
- Confidence score exceeds a predefined threshold (e.g., > 0.7).

4.2 Escalation Criteria

- Confidence score falls below the threshold.
- No relevant chunks are retrieved from the database.
- Query is complex, ambiguous, or outside the knowledge base.
- LLM fails to generate a meaningful response.

---

5. HITL Design

5.1 Escalation Trigger

Escalation is initiated when the system detects low confidence, insufficient context, or inability to generate an appropriate response.

5.2 Post-Escalation Process

- The query along with retrieved context is forwarded to a human agent.
- The human agent analyzes the query and provides a response.

5.3 Response Integration

- The human-generated response is integrated into the system.
- The final response is delivered to the user.
- Optionally, the response can be logged for future system improvement.

---

6. API / Interface Design

6.1 Input Format

```
POST /query
{
```

```
  "question": "What is the return policy?"
}
```

6.2 Output Format

```
{
  "response": "The return policy allows returns within 30 days...",
  "source": "Relevant document reference",
  "escalated": false
}
```

6.3 Interaction Flow

1. The user submits a query via the interface.
2. The query is processed and passed to the LangGraph workflow.
3. Relevant data is retrieved from the vector database.
4. The LLM generates a response.
5. The response is either returned or escalated based on system evaluation.

---

7. Error Handling

7.1 Missing Input Data

- Validate user input before processing.
- Return an appropriate error message for empty or invalid queries.

7.2 No Relevant Chunks Found

- Return a fallback response indicating insufficient information.
- Trigger escalation if necessary.

7.3 LLM Failure

- Implement retry mechanisms for response generation.
- Escalate to HITL if repeated failures occur.

---

8. Conclusion

The LLD provides a **detailed blueprint** for implementing the RAG-based system with:

- Modular architecture
- Structured workflow
- Intelligent decision-making
- HITL integration

This ensures:

- High accuracy
- Scalability
- Robust performance